

JOYFILL RUNTIME FILE MODIFICATION

A Defender XDR hunting model for Node.js writes into developer-tool runtime files after a compromised npm package import.

HUNTING QUERY

CONFIDENCE: MEDIUM

IMPORT-TIME EXECUTION

node



OBSERVED INJECTION TARGETS

- npm/lib/cli.js
- @vscode/deviceid/dist/index.js
- discord_desktop_core/index.js
- GitHub Desktop resources/app/main.js

REPORT DATE

30 July 2026

RESEARCH DATE

28 July 2026

PRIMARY TELEMETRY

Microsoft Defender XDR ·
DeviceFileEvents

OPERATIONAL VALUE

8.4 / 10 · Developer endpoint
priority

Hunting query - Untested implementation sketch. A match is an investigation lead and does not establish malicious modification, credential theft or campaign attribution.



EXECUTIVE ASSESSMENT

Decision-grade summary for Detection Engineering and SOC operations.

JOYFILL · 2026

EXECUTIVE SUMMARY

Hunt the durable file-write boundary, not short-lived package or C2 indicators.

Six malicious Joyfill prerelease packages executed an obfuscated Node.js implant when imported. The implant can rewrite runtime files used by npm and desktop developer tools, creating a repeatable endpoint behavior with stronger durability than package versions, hashes or blockchain endpoints.

OPERATIONAL DECISION

Keep the logic in hunting until local ActionType and updater overlap are measured.

DeviceFileEvents exposes target path and initiating process in one event. A Node.js event against the documented runtime files is high-priority evidence, but legitimate upgrades can overlap and the query does not inspect written content.

8.4

01

OPERATIONAL VALUE / 10

Medium

02

CONFIDENCE

Hunting

03

READINESS

SOURCE FACTS

Observed and extracted

- Payload executes on import, not through postinstall.
- Node.js writes self-reloading blocks to four tool families.
- All six affected versions were detonated and diffed.

ASSESSMENT

Detection value

- Target files are security-sensitive execution surfaces.
- Initiating process attribution narrows the hunt.
- File diff and process pivots can strengthen disposition.

ASSUMPTIONS

Environment controls

- MDE covers developer endpoints and CI runners.
- File events retain Node.js attribution.
- Legitimate update activity can be identified.

Evidence boundary

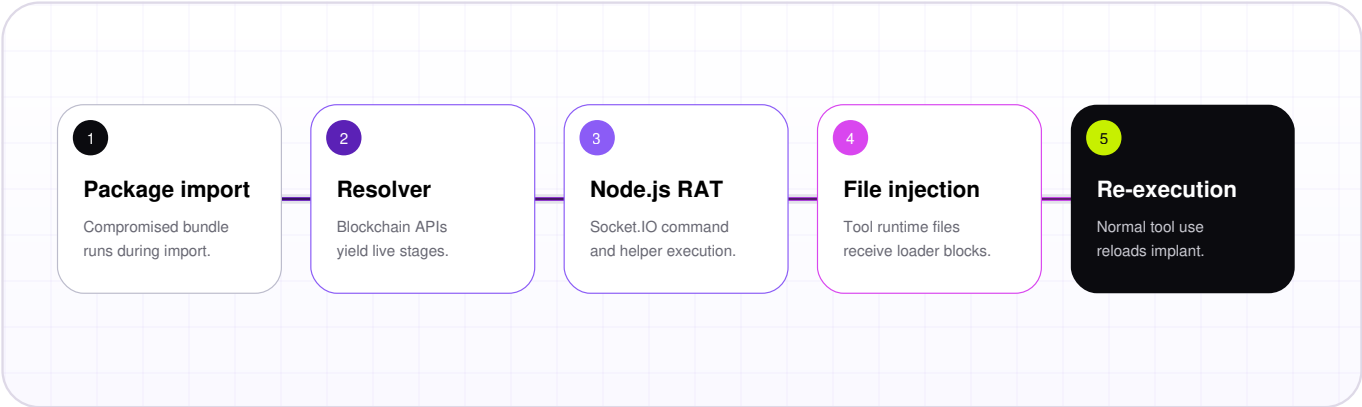
The hunt proves only a recorded file event involving a source-observed target and Node.js. It does not prove file modification, malicious content, package import, persistence or credential theft.

02

THREAT MECHANICS

Source-supported chain from package import to developer-tool persistence.

JOYFILL · 2026



SOURCE EVIDENCE AND OBSERVABILITY

STAGE	CONFIRMED BEHAVIOR	OBSERVABLE EVIDENCE	DETECTION ROLE
Import	Injected bundle runs when the package is imported.	Process and package execution context; not proven by a file event alone.	Context only
Child execution	Detached node -e branch is implemented.	DeviceProcessEvents with process lineage and command line.	Enrichment
Runtime injection	Loader blocks target npm, VS Code/Cursor, Discord and GitHub Desktop files.	DeviceFileEvents: target path and initiating process.	Primary anchor
Credential staging	Python stealer stages under .npm paths.	File/process evidence; filename and archive format vary.	Corroboration
C2	Blockchain resolver and Socket.IO channels are implemented.	Network attribution; live server was not observed in detonation.	Secondary only

Durable behavior

Node.js activity against execution-critical tool files is reusable beyond the six known package versions.

Unconfirmed outcome

The source did not observe outbound C2 during detonation; network success must not be inferred.



TELEMETRY & EVIDENCE MODEL

Minimum data contract and multi-platform implementation boundary.

JOYFILL · 2026

PRIMARY DATA SOURCE

Microsoft Defender XDR · DeviceFileEvents

MDE records file creation, modification and other file-system events with target path, device and initiating-process context. The hunt requires coverage on developer workstations and CI runners.

EVIDENCE BOUNDARY

Do not infer written content from a path match.

Marker comments, injected JavaScript and archive contents require file acquisition or trusted-version comparison. Optional fields can be empty and must not be promotion gates without validation.

REQUIRED FIELD CONTRACT

FIELD	RELIABILITY	PURPOSE	FAILURE CONDITION
Timestamp, DeviceId, DeviceName	Documented	Event identity and endpoint pivot.	Sensor or retention gap.
FileName, FolderPath	Documented	Build the normalized target path.	Unexpected product path/version.
InitiatingProcessFileName	Documented	Bind the event to Node.js.	Write attributed to a helper process.
InitiatingProcessId, InitiatingProcessCreationTime	Documented	Strong local process pivot.	Missing process attribution.
ActionType, hashes, command line	Environment-dependent population	Disposition and enrichment only.	Empty or semantically different values.

REQUIRED

Direct evidence

Device, target file path and initiating Node.js process in the same file event.

OPTIONAL

Enrichment

Process tree, egress, software inventory, package/lockfile exposure and trusted file diff.

UNAVAILABLE

From this table

Injected JavaScript content, marker comments, package import intent and credential material.

Secondary platforms

Splunk requires a verified EDR index, sourcetype and extraction set. Chronicle requires an observed parser mapping for principal process and target file. No SPL or YARA-L rule is published without those prerequisites.

04

DETECTION LOGIC

Primary-platform hunting query and condition rationale.

JOYFILL · 2026

DETECTION HYPOTHESIS

If an attacker uses the Joyfill loader or an equivalent implant to persist in developer-tool runtime files, telemetry should show a Node.js process generating a file event against a documented target. Legitimate activity can be distinguished through action semantics, maintenance context, trusted file diff and surrounding process/network telemetry.

STATUS

Hunting query - Untested implementation sketch

No threshold, rarity model, sequence, join or global allowlist is used. The portal time picker controls lookback. Every match requires investigation.

KQL · HUNTING · UNTESTED

```
// Joyfill-derived Node.js writes to developer-tool runtime files
// Classification: Hunting query - Untested implementation sketch
// Confidence: Medium
// Platform: Microsoft Defender XDR Advanced Hunting
// Required telemetry: DeviceFileEvents from developer workstations and CI runners
//
// Source evidence: the Joyfill implant uses Node.js to inject a self-reloading
// loader into the global npm CLI, @vscode/deviceid, Discord desktop core, and
// GitHub Desktop main.js. This query does not rely on package names, versions,
// hashes, C2 infrastructure, or a specific ActionType value.
DeviceFileEvents
| where InitiatingProcessFileName in~ ("node.exe", "node")
| extend NormalizedPath =
    tolower(replace_string(strcat(FolderPath, "/", FileName), "\\", "/"))
| where
    NormalizedPath endswith "/npm/lib/cli.js"
    or NormalizedPath endswith "@vscode/deviceid/dist/index.js"
    or (
        NormalizedPath contains "/discord_desktop_core"
        and NormalizedPath endswith "/index.js"
    )
    or (
        NormalizedPath endswith "/resources/app/main.js"
        and (
            NormalizedPath contains "/githubdesktop/"
            or NormalizedPath contains "/github desktop.app/"
        )
    )
| project
    Timestamp,
    DeviceId,
    DeviceName,
    ActionType,
    FileName,
    FolderPath,
    SHA1,
    FileSize,
    InitiatingProcessFileName,
    InitiatingProcessFolderPath,
    InitiatingProcessCommandLine,
    InitiatingProcessAccountName,
    InitiatingProcessSHA1,
    InitiatingProcessId,
    InitiatingProcessCreationTime,
    ReportId
| order by Timestamp desc

// Interpretation:
// A result establishes only that a Node.js process generated a recorded file
// event against one of the source-observed developer-tool runtime files.
// Review ActionType and surrounding process/file/network telemetry before
// concluding that the file was modified or that the Joyfill implant executed.
//
// Validate:
// 1. Confirm DeviceFileEvents coverage on developer endpoints and CI runners.
// 2. Record ActionType values and benign matches during legitimate upgrades.
// 3. Inspect the affected file for source-observed marker comments and diff it
//    against a trusted application/package version.
// 4. Pivot on InitiatingProcessId/CreationTime and DeviceId for detached
//    node -e execution, runtime npm installs, and unexpected egress.
```

REVIEW FIELDS · BASELINE LOCALLY · VALIDATE BEFORE PRODUCTION



TRIAGE & INVESTIGATION

Evidence-led workflow for a Node.js runtime-file match.

JOYFILL · 2026

TRIAGE WORKFLOW

- 01

Confirm the raw event.
Review target path, Node.js image, ActionType, process ID and timestamp.
- 02

Identify change context.
Check approved upgrades, package management, CI job and desktop-app updater activity.
- 03

Acquire and diff the file.
Compare against a trusted version and inspect for source-observed marker comments.
- 04

Pivot on process identity.
Use device, process ID and creation time for node -e, child processes and runtime npm installs.
- 05

Corroborate impact.
Review Node.js egress, staged .npm artifacts and credential-access evidence.

ESCALATION LOGIC

Require content or behavioral corroboration.

- **Benign:** expected updater, trusted file diff and approved window.
- **Suspicious:** unexplained write to a target file with incomplete content evidence.
- **Likely malicious:** injected markers or malicious diff plus detached Node.js, staging or C2 evidence.
- **Insufficient:** missing action/process context or unavailable endpoint.

INVESTIGATION PIVOTS

File

Hash, content diff, marker comments, modification time and trusted package baseline.

Process

Node.js path, parent, command line, detached execution and runtime dependency installs.

Host

Developer role, CI runner identity, installed tools and recent maintenance.

Package

Lockfile versions, import/build evidence and clean reinstall history.

Network

Node.js egress to blockchain APIs or unapproved endpoints; never use alone.

Credentials

Git, npm, cloud, Kubernetes, browser and wallet material present on the host.

Response boundary

Do not rotate credentials or isolate a system from a path-only match. Escalate after file content, process lineage or network/staging evidence supports compromise.



FALSE POSITIVES & VALIDATION

Quality gates before promotion beyond hunting.

JOYFILL · 2026

FALSE-POSITIVE MATRIX

SCENARIO	LIKELIHOOD	RISK	TUNING ACTION
Global npm upgrade or repair	High	Med	Validate package-manager process chain, trusted source and maintenance window.
VS Code/Cursor extension or application update	Med	High	Diff the file; tune only a verified updater + path + publisher combination.
Discord or GitHub Desktop self-update	Med	Med	Retain Node.js attribution and confirm expected updater ownership.
Developer packaging or local tool modification	Med	High	Use device role and change ticket; do not suppress developer fleets globally.
Authorized security simulation	Low	High	Tag the test window and preserve the match as validation evidence.

Validation procedure

- Run over 30 days using the portal time picker.
- Classify every ActionType and application/update context.
- Generate a controlled Node.js write to a lab copy of one target file.
- Verify target path, process identity and pivot keys against ground truth.

DECISION CRITERIA

GATE	REQUIRED OUTCOME
Success	Raw event contains device, target and Node.js process with usable pivot keys.
Rejection	Target path or initiating process is absent/unreliable.
Failure	Matches are dominated by updates and cannot be tuned narrowly.
Promotion	Measured volume, documented exclusions, runbook and accountable owner.

Hunting Current state	Candidate After baseline	Production After measured precision	Optimized Feedback loop
---------------------------------	------------------------------------	---	-----------------------------------

Allowlist discipline

Never allowlist node, a developer device group or an entire application path. Exclusions must bind a verified updater/ process chain, exact product path and approved operational context.



ATT&CK, CONCLUSION & REFERENCES

Coverage statement, analytical conclusion and source control.

JOYFILL · 2026

MITRE ATT&CK MAPPING

TECHNIQUE	NAME	SUPPORT	COVERAGE STATEMENT
T1195.002	Compromise Software Supply Chain	Source-confirmed delivery	Context only; the query does not identify package acquisition.
T1059.007	JavaScript/JScript	Source-confirmed Node.js execution	Initiating Node.js process is a direct query condition.
T1546	Event Triggered Execution	Source-confirmed runtime-file persistence	Partial coverage of file events against execution-trigger surfaces.

ANALYST CONCLUSION

A defensible hunt with a clear promotion boundary.

StepSecurity provides direct evidence that the Joyfill implant injects self-reloading code into developer-tool runtime files. Defender XDR provides the strongest practical view because target file and initiating process coexist in DeviceFileEvents. The query is assessed as a Medium-confidence hunting candidate: technically grounded and investigable, but dependent on local ActionType validation and updater baselining before alerting.

REFERENCES

- 1. StepSecurity, “Compromised npm Packages: @joyfill/components and @joyfill/layouts Ship an Obfuscated Remote Access Trojan,” 28 Jul 2026. <https://www.stepsecurity.io/blog/joyfill-npm-supply-chain-compromise>
- 2. Microsoft Learn, “DeviceFileEvents table in the advanced hunting schema,” updated 22 Jun 2026. <https://learn.microsoft.com/en-us/defender-xdr/advanced-hunting-devicefileevents-table>
- 3. MITRE ATT&CK T1195.002, T1059.007 and T1546. <https://attack.mitre.org/>

DOCUMENT CONTROL

VERSION	DATE	STATUS	OWNER
v1.0	30 Jul 2026	Hunting · Untested	Detection Engineering

Residual risk

Alternative target paths, helper-process writes, incomplete sensor coverage or in-memory-only persistence can evade this hunt.

Publication control

No customer data, private identifiers or live credentials are included. URLs are defanged. The HTML source remains private; only this eight-page PDF, the KQL query and references are publishable.